

Smart Lender

Project Description

Smart Lender is a Machine Learning-based loan approval prediction system that predicts whether a loan application will be approved or rejected based on applicant details.

The system uses historical loan data and analyzes factors such as income, loan amount, credit history, education, and property area. Users enter their details through a Flask web application, and the trained model provides the loan eligibility result.

The project uses Python, Flask, Scikit-learn, XGBoost, Pandas, NumPy, and Joblib for model training, preprocessing, and deployment.

Scenarios

Scenario 1:

A customer applies for a loan by entering income details, credit history, loan amount, and other personal information. The system analyzes the details and predicts loan approval probability, helping the customer understand their eligibility.

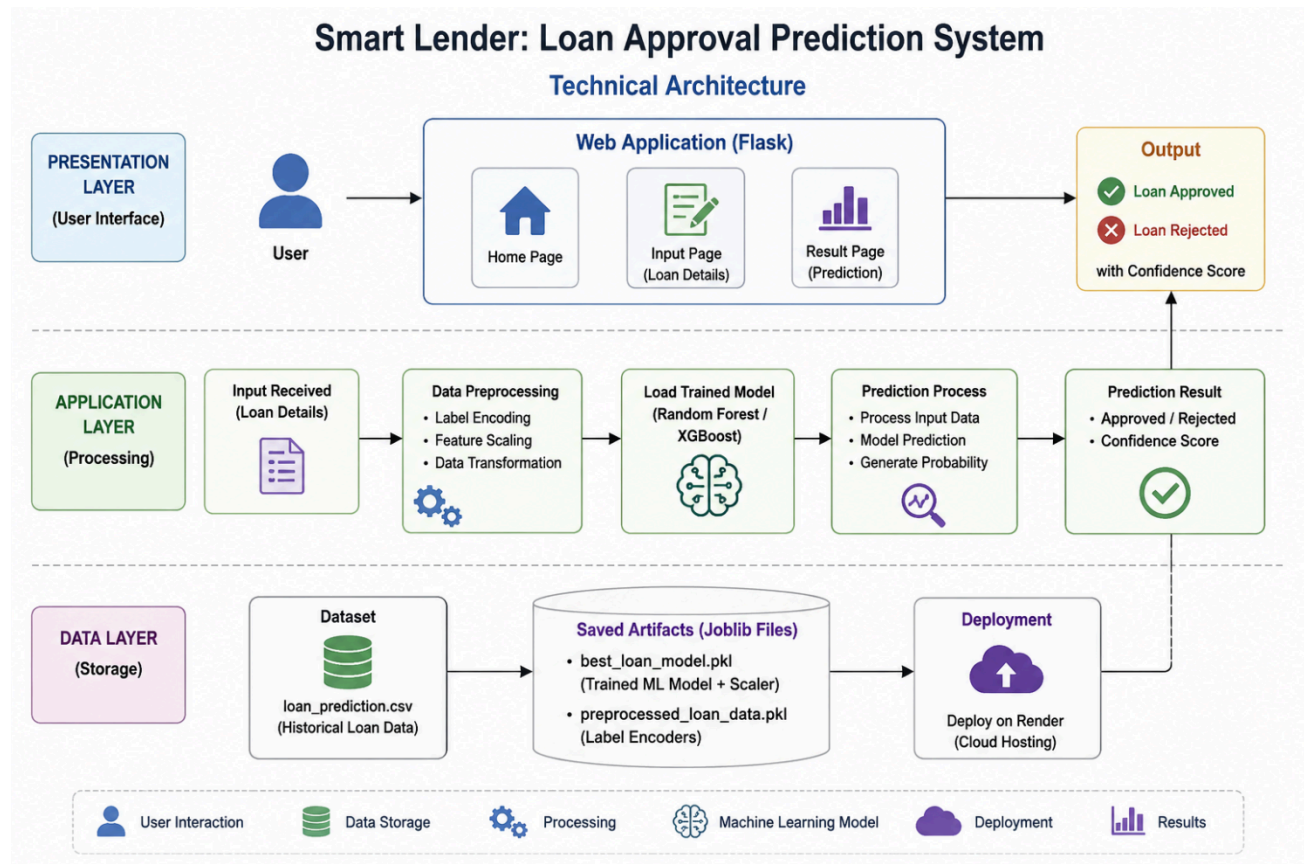
Scenario 2:

A bank employee enters applicant information before processing a loan request. The application predicts whether the loan is likely to be approved or rejected, reducing manual evaluation time.

Scenario 3:

A user compares different financial conditions by changing income, loan amount, or credit history values. The system provides updated predictions, helping the user understand factors affecting loan approval.

Technical Architecture



Architecture Description:

Smart Lender follows a three-layer architecture. The presentation layer consists of a Flask-based web interface developed using HTML and CSS where users enter loan application details. The application layer performs data preprocessing, feature transformation, and executes the trained Machine Learning model to predict loan approval status. The data layer contains the historical loan dataset and saved model files used for generating predictions.

Pre-requisites

- Python Programming
- Flask Framework
- Machine Learning concepts
- Scikit-learn
- XGBoost
- Pandas
- NumPy
- Joblib
- HTML and CSS
- Git and GitHub

Project Workflow

Milestone 1: Dataset Collection and Environment Setup

Activity 1.1: Collect Loan Dataset

Collect historical loan application dataset containing:

- Applicant details
- Income information
- Loan details
- Credit history
- Property information
- Loan approval status

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0	Urba
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0	Rur
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0	Urba
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0	Urba
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0	Urba
...	...	...	...	...	...	...	...	...	...	...	...	...
609	LP002978	Female	No	0	Graduate	No	2900	0.0	71.0	360.0	1.0	Rur
610	LP002979	Male	Yes	3+	Graduate	No	4106	0.0	40.0	180.0	1.0	Rur
611	LP002983	Male	Yes	1	Graduate	No	8072	240.0	253.0	360.0	1.0	Urba
612	LP002984	Male	Yes	2	Graduate	No	7583	0.0	187.0	360.0	1.0	Urba
613	LP002990	Female	No	0	Graduate	Yes	4583	0.0	133.0	360.0	0.0	Semiurba

614 rows × 13 columns

Activity 1.2: Setup Development Environment

Install Python and configure the development environment using VS Code.

Install required libraries:

- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Flask
- Joblib

```

requirements.txt
1 asttokens==3.0.1
2 blinker==1.9.0
3 click==8.4.2
4 colorama==0.4.6
5 comm==0.2.3
6 contourpy==1.3.3
7 cyclical==0.12.1
8 debugpy==1.8.21
9 decorator==5.3.1
10 executing==2.2.1
11 Flask==3.1.3
12 fonttools==4.63.0
13 imbalanced-learn==0.14.2
14 ipykernel==7.3.0
15 ipython==9.14.1
16 ipython_pygments_lexers==1.1.1
17 itsdangerous==2.2.0
18 jedi==0.20.0
19 Jinja2==3.1.6
20 joblib==1.5.3
21 Jupyter_Client==8.9.1
22 jupyter_core==5.0.1
23 kiwisolver==1.5.0
24 MarkupSafe==3.0.3
25 matplotlib==3.11.0
26 matplotlib-inline==0.2.2
27 narwhals==2.22.1
28 nest_asyncio==1.7.2
29 numpy==2.5.0
30 packaging==26.2
31 pandas==3.0.3
32 parso==0.8.7
  
```

```

requirements.txt
32 parso==0.8.7
33 pillow==12.2.0
34 platformdirs==4.10.0
35 prompt_toolkit==3.0.52
36 psutil==7.2.2
37 pure_eval==0.2.3
38 Pygments==2.20.0
39 pyparsing==3.3.2
40 python-dateutil==2.9.0.post0
41 pyzmq==27.1.0
42 scikit-learn==1.9.0
43 scipy==1.18.0
44 seaborn==0.13.2
45 six==1.17.0
46 sklearn-compat==0.1.6
47 stack-data==0.6.3
48 threadpoolctl==3.6.0
49 tornado==6.5.7
50 traitlets==5.15.1
51 typing_extensions==4.15.0
52 tzdata==2026.2
53 wcwidth==0.8.1
54 Werkzeug==3.1.8
55 xgboost==3.3.0
56 gunicorn
  
```

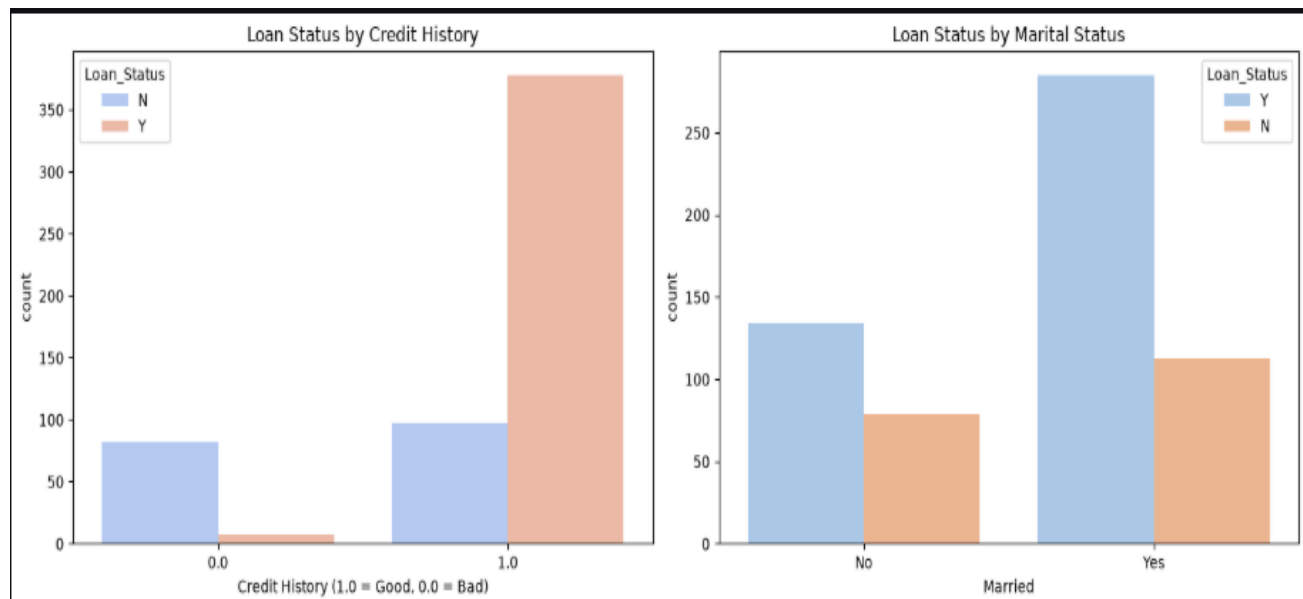
Activity 1.3: Dataset Analysis

Analyze the dataset using Pandas.

Perform:

- Missing value analysis
- Statistical analysis
- Feature distribution analysis
- Relationship analysis between features

```
--- Missing Values Count per Column ---
Gender          13
Married          3
Dependents       15
Self_Employed   32
LoanAmount       22
Loan_Amount_Term 14
Credit_History  50
dtype: int64
```



Activity 1.4: Define System Architecture

Design the interaction between:

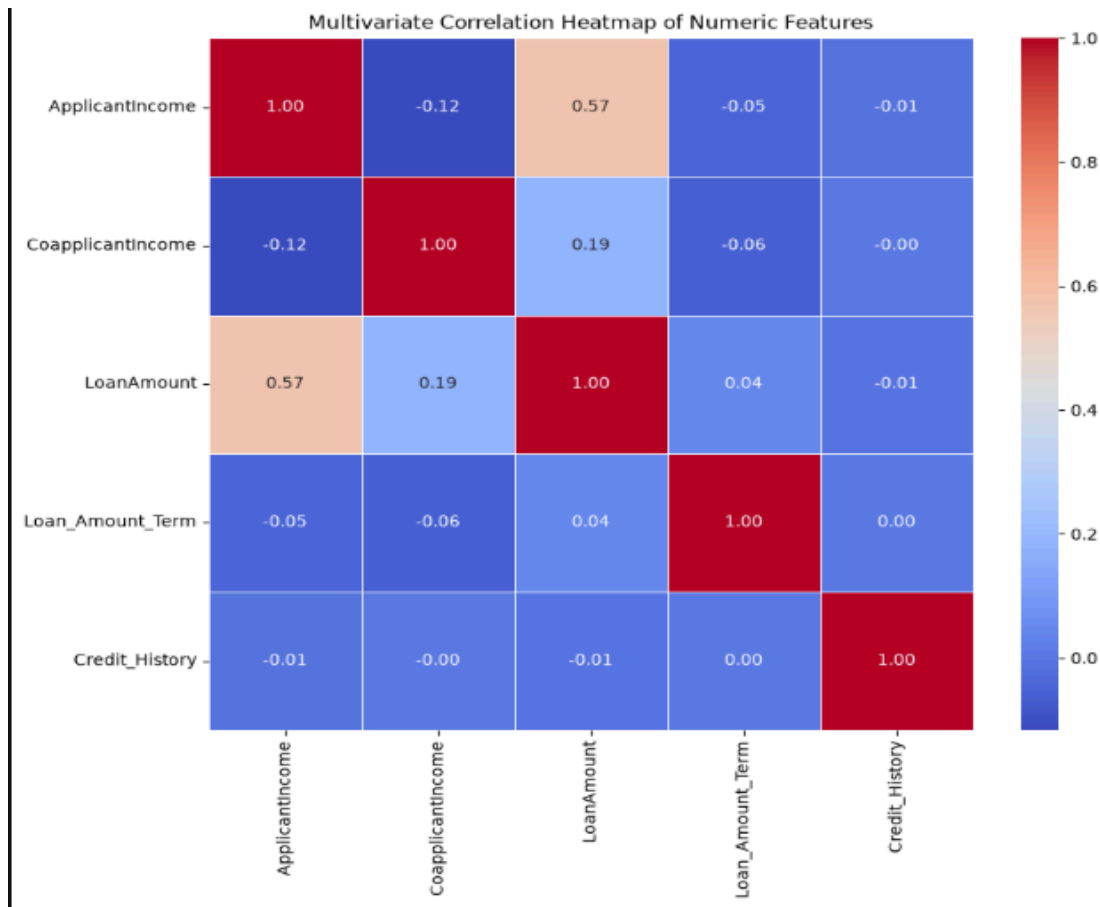
- Flask application
- Machine Learning model
- Preprocessing module
- Loan dataset

Milestone 2: Data Analysis and Preprocessing

Activity 2.1: Data Visualization

Perform:

- Histogram analysis
- Correlation analysis
- Feature distribution analysis
- Data visualization



Activity 2.2: Data Cleaning

Prepare the dataset by:

- Handling missing values
- Encoding categorical variables
- Removing unnecessary features
- Selecting important features

```
# Drop Loan_ID since it's an arbitrary tracking string, not a pattern
if 'Loan_ID' in df.columns:
    df = df.drop(columns=['Loan_ID'])

# List of text columns that need converting to numbers
categorical_cols = [
    'Gender',
    'Married',
    'Dependents',
    'Education',
    'Self_Employed',
    'Property_Area',
    'Loan_Status'
]

# Create a dictionary to store one encoder per column
label_encoders = {}

# Encode each categorical column and save its encoder
for col in categorical_cols:
    encoder = LabelEncoder()
    df[col] = encoder.fit_transform(df[col].astype(str))
    label_encoders[col] = encoder

print("Categorical columns encoded successfully!")
df.head()
```

Activity 2.3: Feature Scaling

Split dataset into training and testing data.

Apply preprocessing techniques:

- Label Encoding
- Standard Scaling

```
--- Data Preprocessing Complete ---
Training features shape : (675, 11)
Testing features shape  : (169, 11)
Training labels count   : {1: 338, 0: 337}
Testing labels count    : {0: 85, 1: 84}
```

Milestone 3: Model Building

Activity 3.1: Train Machine Learning Models

Train and compare:

- Decision Tree
- Random Forest

- K-Nearest Neighbor
- XGBoost

```
# 1. Load the preprocessed data arrays
data = joblib.load('models/preprocessed_loan_data.pkl')
X_train = data['X_train']
X_test = data['X_test']
y_train = data['y_train']
y_test = data['y_test']

print("Data loaded successfully! Ready to train models.\n")

# 2. Initialize the specific models required by your Epic 4 spec
models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "KNN Model": KNeighborsClassifier(),
    "XGBoost Model": xgb.XGBClassifier(random_state=42, eval_metric='logloss')
}

# 3. Train and print performance metrics
for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    accuracy = accuracy_score(y_test, predictions)
    print(f"=== {name} ===")
    print(f"Accuracy Score: {accuracy * 100:.2f}%")
    print(classification_report(y_test, predictions))
    print("-" * 40)
```

Activity 3.2: Model Evaluation

Evaluate models using:

- Accuracy Score
- Confusion Matrix
- Classification Report

Select the best performing model.

```
Data loaded successfully! Ready to train models.

=== Decision Tree ===
Accuracy Score: 78.70%
      precision    recall  f1-score   support

         0         0.78        0.81        0.79         85
         1         0.80        0.76        0.78         84

   accuracy          0.79
  macro avg          0.79
 weighted avg          0.79

-----
=== Random Forest ===
Accuracy Score: 82.25%
      precision    recall  f1-score   support

         0         0.86        0.78        0.81         85
         1         0.79        0.87        0.83         84

   accuracy          0.82
  macro avg          0.83
 weighted avg          0.83
  ...
  macro avg          0.85
 weighted avg          0.85

-----

🏆 THE WINNER IS: XGBoost Model with an Accuracy of 84.62%
```

Activity 3.3: Save Best Model

Save:

best_loan_model.pkl

preprocessed_loan_data.pkl

using Joblib for deployment.

```
joblib.dump(preprocessing_bundle, 'models/preprocessed_loan_data.pkl')

# Save it to disk
joblib.dump(model_artifacts, 'models/best_loan_model.pkl')
```

▼ models

- best_loan_model.pkl
- preprocessed_loan_data.pkl

Milestone 4: Flask Web Application Development

Activity 4.1: Develop Flask Backend

Create app.py to:

- Load trained model
- Load preprocessing objects
- Receive user inputs
- Perform preprocessing
- Generate prediction
- Display result


```

app.py > input_page
8
9 import logging
10 from flask import Flask, render_template, request
11 from predict import predict_loan
12
13 # =====
14 # APPLICATION CONFIGURATION
15 # =====
16
17 app = Flask(__name__)
18
19 # Configure logging
20 logging.basicConfig(
21     level=logging.INFO,
22     format="%(asctime)s | %(levelname)s | %(message)s"
23 )
24
25 # =====
26 # HOME PAGE
27 # =====
28
29 @app.route("/")
30 def home():
31     return render_template("home.html")
32
33 # =====
34 # INPUT PAGE
35 # =====
36
37
38 @app.route("/input")
39 def input_page():
40     return render_template("input.html")
41
42 # =====
43 # PREDICTION ROUTE
44 # =====
45
46
47 @app.route("/predict", methods=["POST"])
48 def predict():
49
50     try:
51
52         # Get user inputs
53         form_data = request.form.to_dict()
54
55
56         logging.info("Received loan application.")
57
58         # Predict loan status
59         result, confidence = predict_loan(form_data)
60
61         # Convert prediction into user-friendly text
62         if result == "Y":
63             prediction = "Loan Approved"
64         else:
65             prediction = "Loan Rejected"
66
67         logging.info(
68             f"Prediction completed successfully. Result: {prediction}"
69         )
70
71         return render_template(
72             "output.html",
73             prediction=prediction,
74             confidence=confidence
75         )
76
77     except ValueError as e:
78
79         logging.error(f"Input Error: {e}")
80
81         return render_template(
82             "output.html",
83             prediction="Input Error",
84             confidence=0,
85             error=str(e)
86         ), 400
87
88     except FileNotFoundError as e:
89
90         logging.error(f"Model File Missing: {e}")
91
92         return render_template(
93             "output.html",
94             prediction="System Error",
95             confidence=0,
96             error="Model files could not be found."
97         ), 500
98
99     except Exception as e:
100
101         logging.exception("Unexpected Error")
102
103         return render_template(
104             "output.html",
105             prediction="Prediction Failed",
106             confidence=0,
107             error=str(e)
108         ), 500
109
110 # =====
111 # START APPLICATION
112 # =====
113
114 if __name__ == "__main__":
115
116     logging.info("Starting Smart Lender Application...")
117
118     app.run(
119         host="0.0.0.0",
120         port=5000
121     )
122

```

Activity 4.2: Develop User Interface

Create pages:

- Home Page



SMART LENDER

Smart Loan Eligibility Checker

Welcome to **Smart Lender**, your trusted digital assistant for assessing loan eligibility quickly and conveniently.

By providing your personal and financial information, you can receive an instant assessment indicating whether your loan application is likely to meet the eligibility criteria.

This assessment is designed to help applicants make informed decisions before proceeding with a formal loan application. The final approval remains subject to the lending institution's verification process, documentation, and applicable lending policies.

Why Choose Smart Lender?

✓ Quick Eligibility Assessment	Receive an instant loan eligibility result.
✓ Simple & User-Friendly	Complete the assessment in just a few easy steps.
✓ Secure Information Processing	Your submitted information is processed securely for assessment.
✓ Instant Results	View your eligibility assessment immediately after submission.
✓ Confidence Score	Each assessment includes a confidence score for better understanding.

Disclaimer: The assessment provided by Smart Lender is intended for informational purposes only and should not be considered a final lending decision. Actual loan approval is subject to document verification, credit evaluation, and the lending institution's policies.

[Check Loan Eligibility →](#)

© 2026 Smart Lender | Helping You Make Better Financial Decisions

- Loan Input Page



Smart Lender

Loan Application Form

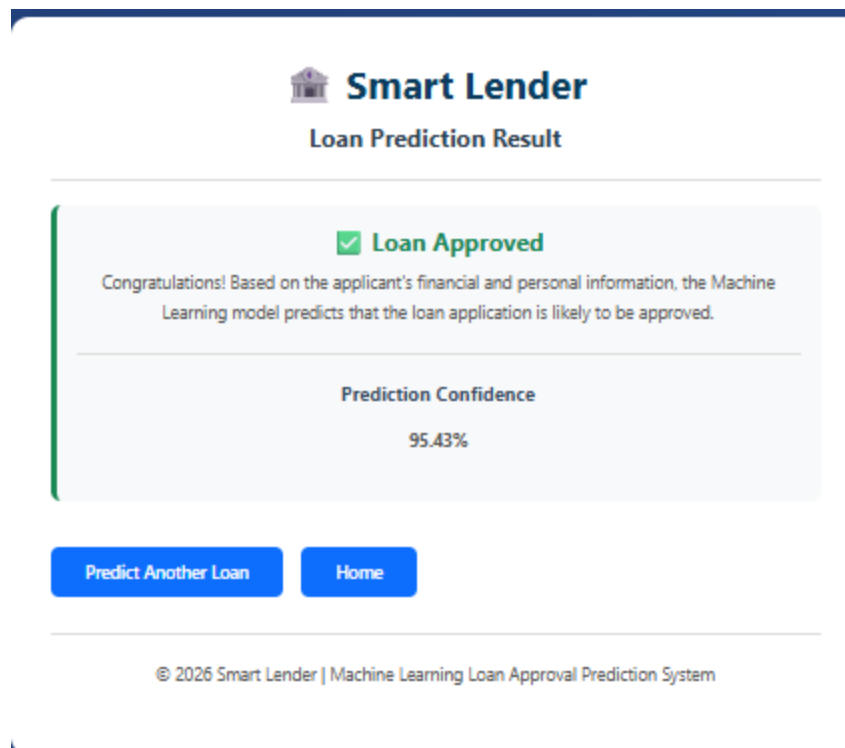
Enter the applicant's information below to predict whether the loan application is likely to be approved.

Gender	-- Select Gender --
Marital Status	-- Select Marital Status --
Dependents	-- Select Dependents --
Education	-- Select Education --
Self Employed	-- Select Option --
Applicant Income	Enter Applicant Income
Co-applicant Income	Enter Co-applicant Income
Loan Amount	Enter Loan Amount
Loan Term (Months)	Example: 360
Credit History	-- Select Credit History --
Property Area	-- Select Property Area --

[Predict Loan Approval](#)

Smart Lender | Machine Learning Loan Approval Prediction System

- Prediction Result Page



Using:

- HTML
- CSS
- Flask Templates

Activity 4.3: Integrate Machine Learning Model

Connect the trained model with Flask.

User input → preprocessing → model prediction → output display

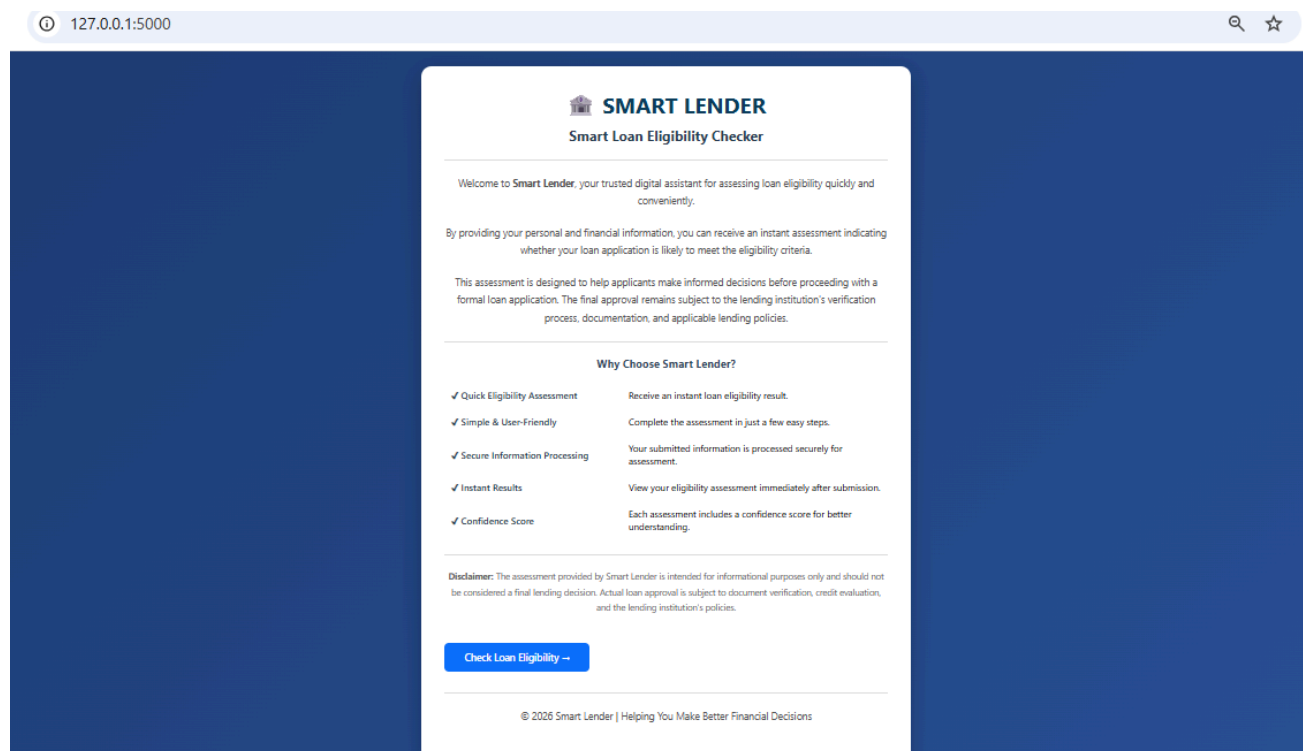
Milestone 5: Testing and Deployment

Activity 5.1: Local Testing

Verify:

- Input validation
- Prediction generation
- Page navigation
- Output display

```
PS C:\Users\HP\Desktop\Smart Lender> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\HP\Desktop\Smart Lender\venv\Scripts\Activate.ps1")
(venv) PS C:\Users\HP\Desktop\Smart Lender> python app.py
2026-07-01 14:06:17,609 | INFO | Starting Smart Lender Application...
* Serving Flask app 'app'
* Debug mode: off
2026-07-01 14:06:18,962 | INFO | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.29.175:5000
2026-07-01 14:06:18,963 | INFO | Press CTRL+C to quit
```



Activity 5.2: Model Verification

Test the application using different applicant details.

Check:

- Approved cases
- Rejected cases
- Confidence score

Activity 5.3: Deployment

Prepare deployment by:

- Creating requirements.txt
- Organizing project files

- Creating README.md
- Uploading project to GitHub
- Deploying using Render

WEB SERVICE

Smart-Lender

Python 3

Free

[Upgrade your instance →](#)

Connect ▾

Manual Deploy ▾

Service ID: `srv-d915e8v7f7vs73d5cvg0`  Lalasa-web / Smart-Lender-  main<https://smart-lender-i9fg.onrender.com>  smart-lender-i9fg.onrender.com   **SMART LENDER**
Smart Loan Eligibility Checker

Welcome to **Smart Lender**, your trusted digital assistant for assessing loan eligibility quickly and conveniently.

By providing your personal and financial information, you can receive an instant assessment indicating whether your loan application is likely to meet the eligibility criteria.

This assessment is designed to help applicants make informed decisions before proceeding with a formal loan application. The final approval remains subject to the lending institution's verification process, documentation, and applicable lending policies.

Why Choose Smart Lender?

✓ Quick Eligibility Assessment	Receive an instant loan eligibility result.
✓ Simple & User-Friendly	Complete the assessment in just a few easy steps.
✓ Secure Information Processing	Your submitted information is processed securely for assessment.
✓ Instant Results	View your eligibility assessment immediately after submission.
✓ Confidence Score	Each assessment includes a confidence score for better understanding.

Disclaimer: The assessment provided by Smart Lender is intended for informational purposes only and should not be considered a final lending decision. Actual loan approval is subject to document verification, credit evaluation, and the lending institution's policies.

[Check Loan Eligibility →](#)

© 2026 Smart Lender | Helping You Make Better Financial Decisions